

Evaluación del rendimiento de un servidor de Minecraft en Windows Server y Ubuntu Server

Ignacio Caballero Román
10 horas

José Manuel González Escobar
8 horas

Sergio Alcántara Áviles
8 horas

Pablo Cáceres Gaitán
6 horas

10 de diciembre de 2023

Contents

1	Apéndice	4
2	Instalacion e información	4
2.1	Información sobre la máquina	4
2.2	Instalación de los sistemas operativos	5
2.2.1	Ubuntu Server	5
2.2.2	Windows Server	6
2.3	Red virtual	6
2.4	Primeras impresiones	7
2.5	Dependencias	7
3	Configuración de Ubuntu Server	8
3.1	Conexión a la máquina virtual	8
3.2	Instalación de las dependencias y configuración	8
4	Configuración de Windows Server	10
4.1	Instalación de Java 1.8	10
4.2	Instalación del servidor de Minecraft	10
4.3	Firewall de Windows	11
5	Benchmark	13
5.1	Resultados	14
5.1.1	Uso de CPU	15
5.1.2	Uso de memoria RAM	16
5.1.3	Número de peticiones	17
5.1.4	Duración del benchmark	17
5.2	Análisis de los datos	18

1 Apéndice

Para la realización vamos a comparar el rendimiento que va a tener el servidor de Minecraft que hemos decidido montar en una máquina virtual ejecutando Windows Server comparado con una solución de Ubuntu Server

2 Instalacion e información

2.1 Información sobre la máquina

Para el desarrollo de este trabajo, hemos optado en usar máquinas virtuales para realizar las pruebas necesarias, las cuales se van a ejecutar en un portátil Fujitsu Livebook A544 ejecutando un sistema operativo Debian limpio como base para evitar que los resultados de los *benchmark* no sean afectados por software como un entorno de escritorio u otros programas.

Las especificaciones técnicas de la máquina on las siguientes:

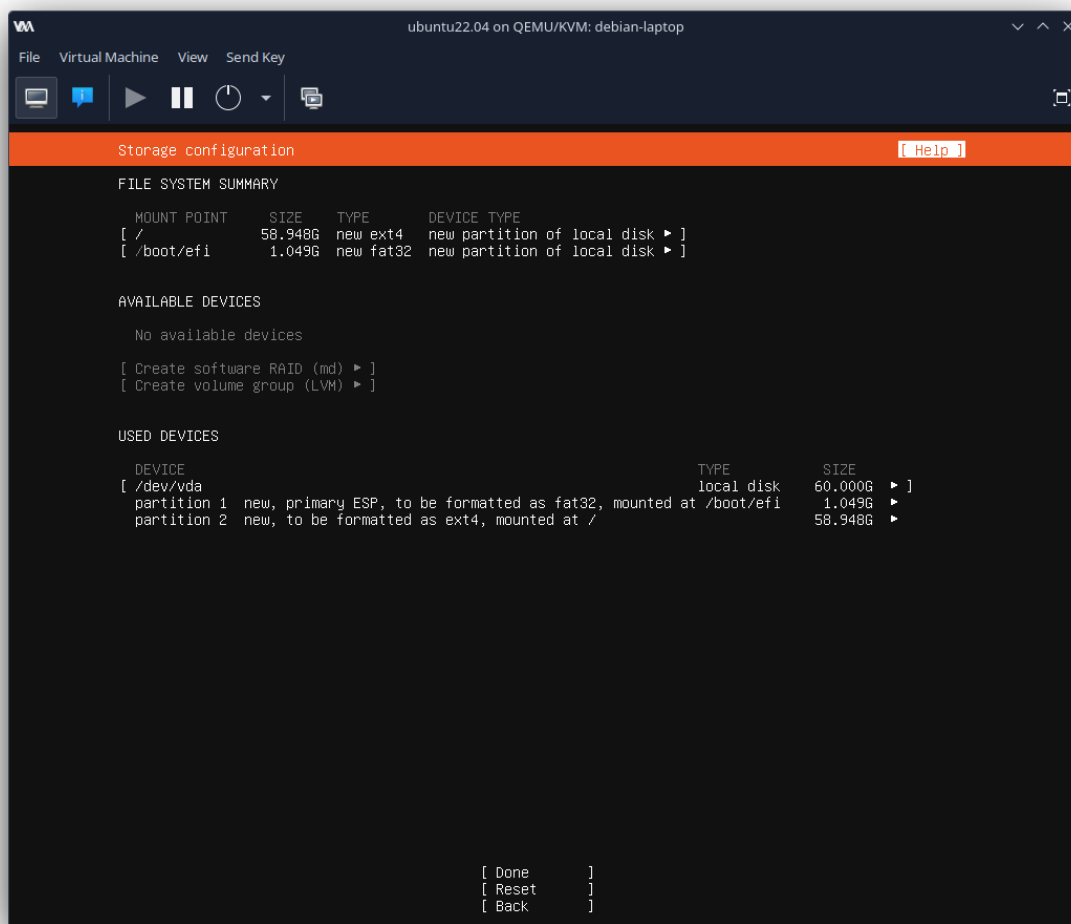
- Intel Core i5 4200M (2 núcleos y 4 hilos a 3 GHz)
- 12 GB de RAM DDR3 a 1600 MHz
- Disco duro mecánico de 250 GB marca Seagate

Para crear las máquinas virtuales vamos a utilizar virtio. Virtio, al mismo tiempo que VirtualBox o VMWare player, es un hypervisor de tipo 2 código abierto que trabaja en tándem con el núcleo de Linux, proporcionando así un sistema de virtualización asistida por el kernel (KVM) que nos permite crear máquinas virtuales por medio de QEMU, y nos proporciona una ventaja clave, ya que podemos trabajar con el de forma remota, así no necesitamos tener instalada una interfaz gráfica en el servidor la cual pueda afectar al resultado de nuestras pruebas.

Los ajustes de la máquina virtual para la configuración de la red será por medio de la NAT interna de la máquina virtual, debido a las limitaciones de este portátil, pero haciendo uso de *port forwarding* podemos acceder a los servicios que se están ejecutando dentro de la máquina.

2.2 Instalación de los sistemas operativos

2.2.1 Ubuntu Server

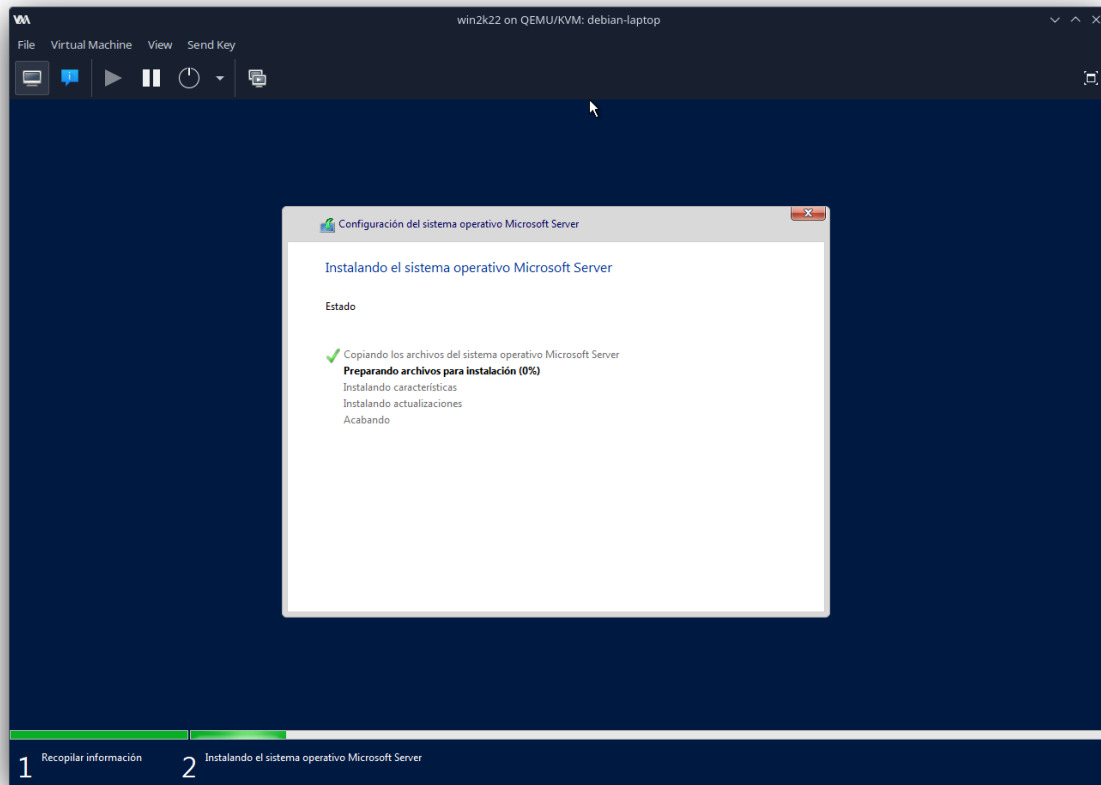


Para la instalación de Ubuntu Server 22.04.3 LTS hemos recurrido a usar una imagen de disco virtual de 60 GB, lo cual es más que suficiente para la instalación del sistema operativo y la ejecución de un servidor de Minecraft, además de las dependencias necesarias para la instalación del mismo.

Para optimizar recursos, hemos intentado reducir al máximo la cantidad de paquetes que se instalarán en el servidor, así que hemos seleccionado durante la instalación de optar por una instalación minimalista.

Para el esquema de particiones, hemos decidido mantenerlo simple haciendo uso de dos particiones, una partición para el sistema EFI y otra para el sistema operativo.

2.2.2 Windows Server



Para Windows Server, hemos utilizado Windows Server 2022 Standard Edition y hemos aplicado las mismas características de la máquina virtual de Ubuntu para que sea una comparación justa, esto nos lleva a tener que instalar los drivers para la máquina virtual, ya que en Ubuntu Server estos vienen incluidos por defecto en el kernel de Linux y no queremos alterar nada del *hardware* virtual entre las dos máquinas, ya que eso podría generar discrepancias a la hora de la comparación.

Se ha instalado con experiencia de escritorio, ya que interactuar con Windows Server usando la consola puede resultar muy tedioso y excesivamente complicado. Es verdad que esto podría generar un uso mayor de recursos ya que tiene que estar ejecutándose el explorador de Windows como una tarea de fondo, pero es mucho mejor a tener que utilizar la línea de comandos de Windows.

2.3 Red virtual

Debido a las limitaciones del sistema que estamos utilizando para realizar esta evaluación, es necesario hacer una apertura de puertos en el sistema para que tengamos acceso a los distintos servicios de las máquinas virtuales desde el exterior. Para ello, he usado los comandos de iptables

correspondientes para abrir los puertos y luego de eso hemos hecho pruebas con el servicio de SSH.

A continuación se muestran las reglas utilizadas para permitir las conexiones entrantes a la máquina virtual.

```
-A PREROUTING -d 172.16.12.134/32 -p udp -m udp --dport 5901 -j DNAT --to-destination 192.168.122.26:5901
-A PREROUTING -d 172.16.12.134/32 -p tcp -m tcp --dport 5901 -j DNAT --to-destination 192.168.122.26:5901
-A PREROUTING -d 172.16.12.134/32 -p udp -m udp --dport 25566 -j DNAT --to-destination 192.168.122.26:25566
-A PREROUTING -d 172.16.12.134/32 -p tcp -m tcp --dport 25566 -j DNAT --to-destination 192.168.122.26:25566
-A PREROUTING -d 172.16.12.134/32 -p udp -m udp --dport 25565 -j DNAT --to-destination 192.168.122.29:25565
-A PREROUTING -d 172.16.12.134/32 -p tcp -m tcp --dport 25565 -j DNAT --to-destination 192.168.122.29:25565
-A PREROUTING -d 172.16.12.134/32 -p tcp -m tcp --dport 2222 -j DNAT --to-destination 192.168.122.29:22
```

Además de esto, en cada máquina correspondiente, se ha permitido en su firewall correspondiente las reglas necesarias para las aplicaciones correspondientes.

2.4 Primeras impresiones

Tras haber realizado la instalación de ambos sistemas operativos, podemos empezar a comparar algunos aspectos entre ambos, por ejemplo, Windows hace mayor uso de recursos como memoria RAM y realiza más operaciones de entrada y salida que Ubuntu Server, pero al mismo tiempo, este es el sistema operativo que más características trae.

Este ultimo factor puede ser bueno o malo dependiendo de la actividad que vayamos a realizar. En el caso de una gran empresa que tengo que ejecutar diversas aplicaciones podría resultar interesante, pero en el caso de ejecutar un servidor de juegos, podría ser causante de *bottlenecks* e impactar de forma negativa la experiencia de los usuarios conectados.

2.5 Dependencias

Ya que el objetivo final de esta práctica va a ser evaluar el rendimiento de un servidor de Minecraft tanto en un servidor de Windows como en uno de Linux, hemos de instalar las distintas herramientas necesarias para evaluar el rendimiento y las dependencias para ejecutar dicho servidor de Minecraft.

En el caso de Windows vamos a usar las herramientas que vienen proporcionadas en el sistema operativo, como el Administrador de tareas para hacer una monitorización por encima,

al mismo tiempo que un programa que trae windows instalado por defecto llamado *perfmom.msc*.

En el caso de Linux también vamos a hacer uso de herramientas que vienen incluidas en el sistema operativo además de otras como por ejemplo el monitor de sar usando un intervalo de muestreo bajo, ya que la carga no se ejecutará a lo largo de todo el día.

Al mismo tiempo, se han instalado las dependencias de Java necesarias para la ejecución de un servidor de Minecraft en la version 1.12.2

3 Configuración de Ubuntu Server

3.1 Conexión a la máquina virtual

Para conectarnos a la máquina virtual primero lanzamos un enlace mediante la aplicación de TunSafe para acceder a la red donde se encuentra dicha máquina, una vez conectados a la red del equipo hacemos un:

```
ssh -p puerto usuario@direccion_ip
```

3.2 Instalación de las dependencias y configuración

Lo primero que tenemos que hacer es instalar los paquetes necesarios para su creacion y configuracion, para esto usamos los comandos:

```
sudo apt update sudo apt upgrade sudo apt install openjdk-8-jre -y
```

Después debemos habilitar el firewall para permitir el tráfico que ingrese por el puerto que deseamos para el servidor, en nuestro caso el 25565 hacia nuestro servidor de Minecraft.

```
sudo ufw allow 25565
```

Una vez hecho esto creamos una carpeta llamada servidor_minecraft con `sudo mkdir servidor_minecraft` y descargamos la versión más reciente de Spigot Server y subimos el archivo a nuestro servidor. Para pasar el archivo de nuestro sistema original al servidor en la máquina virtual, ya que no tenemos una interfaz gráfica, usamos la aplicación de FileZilla y aprovechamos el servidor SFTP que viene activo en el paquete de OpenSSH.

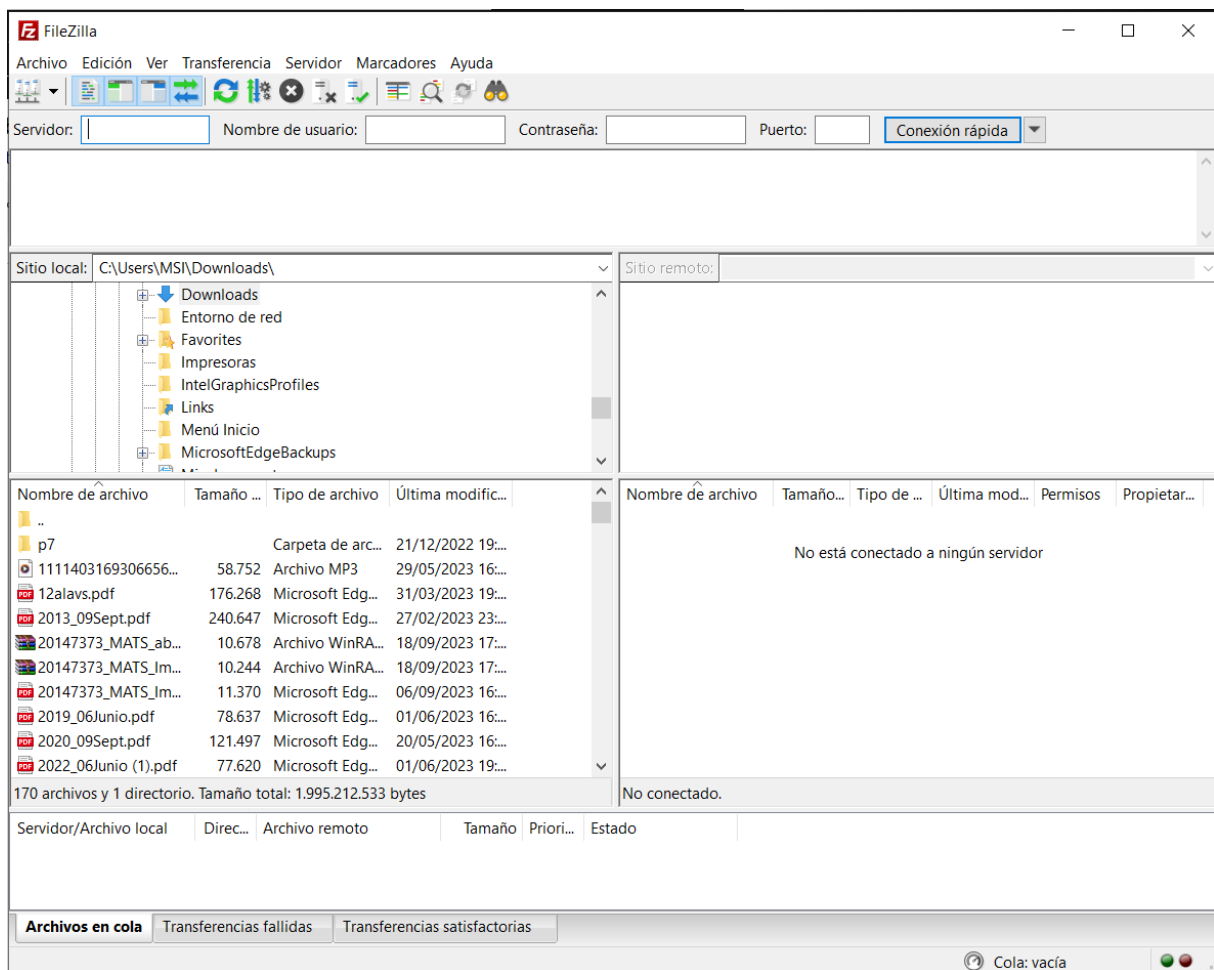


Figure 1: Interfaz gráfica de FileZilla

Simplemente se coloca los detalles sobre la máquina a la que queremos enviar los archivos y los arrastramos dentro de la carpeta que hemos creado con anterioridad.

Una vez el spigot.jar este en el servidor usaremos el siguiente comando para iniciarlo:

```
java -Xms1024M -Xmx1024M -jar spigot.jar nogui
```

Donde:

- Xms1024M: Configura el servidor para comenzar a ejecutarse con 1 GB de RAM. Se puede aumentar este límite si se quiere (Usando M para megabytes y G para gigabytes).
- Xmx1024M: Configura el servidor para usar, como máximo, 1GB de RAM. Se puede aumentar este límite si se quiere. jar: este indicador especifica el archivo jar del servidor que se ejecutará.
- nogui: indica al servidor que no inicie un GUI, ya que este es un servidor y no tiene una interfaz de usuario gráfica.

Para no repetir el mismo comando cada vez que se vaya a iniciar el servidor de Minecraft, es posible crear un script (en nuestro caso `iniciar.sh`) con el comando. luego hay que dar permisos al script con `chmod u+x iniciar.sh`.

la primera vez que se use el comando (o el script con `./iniciar.sh`) fallará al iniciarse pero se generarán 2 archivos nuevos:

- *eula.txt*: es el motivo por el que nos falla al iniciar, tendremos que aceptar el eula. Para ello editamos el archivo con `nano eula.txt` y cambiamos el parámetro `eula` de `false` a `true`.
- *server.properties*: contiene la configuración del servidor como puede ser la dificultad, el número máximo de jugadores, el nombre del servidor, junto con otros detalles, se puede configurar como se quiera, en nuestro caso aumentamos el número de jugadores y permitimos a jugadores no premium.

Además, para poder realizar los benchmarks, hemos tenido que desactivar una opción dentro de la configuración del propio servidor Spigot, ya que esta interfiere con la cantidad máxima de cuentas que se pueden conectar desde una dirección IP.

El archivo en concreto que hemos modificado se llama *bukkit.yml* y hemos cambiado la opción `connection-throttle`, poniendo su valor a `-1`, lo cual evitará que el servidor limite las conexiones por IP.

Con esto listo ya podemos volver a iniciar el servidor y ahora si se ejecuta de forma correcta, teniendo ya la configuración lista para empezar las pruebas de estrés.

4 Configuración de Windows Server

4.1 Instalación de Java 1.8

Para ejecutar un servidor de minecraft lo primero es instalar Java. Para esta versión de Minecraft Server (1.12.2), debemos instalar Java 8.

Para ello, debemos descargar el instalador de la página oficial. Una vez descargado, ejecutamos y seguimos las instrucciones.

4.2 Instalación del servidor de Minecraft

Para instalar el servidor, vamos a hacer una copia del que ya tenemos en Ubuntu Server con las mismas configuraciones con el objetivo de evitar anomalías en el benchmarking. Ya que el servidor de Minecraft es una aplicación hecha en Java, podemos descartar el problema de las plataformas, ya que es independiente de la misma mientras tenga preinstalada la máquina virtual de Java.

Para la ejecución, como el comando es el mismo en Linux que en Windows, simplemente renombramos el `iniciar.sh` a `iniciar.bat`.

4.3 Firewall de Windows

Debido a las restricciones que tiene Windows sobre las conexiones entrantes, hay que añadir una excepción al firewall para poder permitir las conexiones externas de los clientes al servidor de Minecraft.

Sabemos que Minecraft usa por defecto los puertos 25565 tanto en TCP como UDP, por lo que tenemos que añadir una excepción al firewall en esos puertos específicos. Para ello, buscamos el panel de control del firewall que tiene de nombre *Windows Defender Firewall con seguridad avanzada*.

A continuación, debemos crear una nueva regla para permitir la entrada/salida de tráfico por TCP y UDP por el puerto 25565. Este proceso lo repetiremos para la sección de Entrada y Salida y para TCP y UDP:

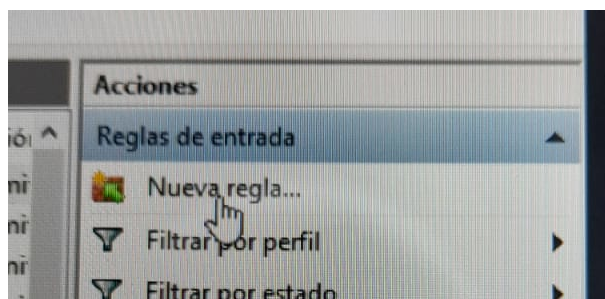
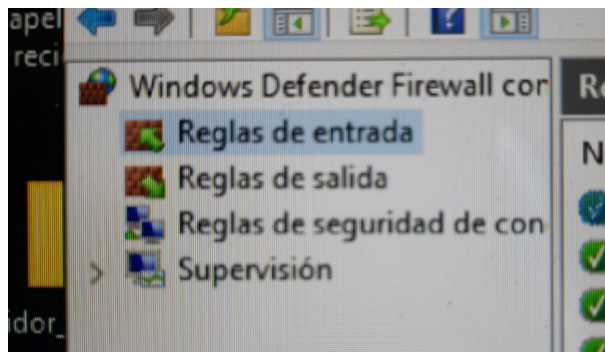


Figure 2: Nueva regla

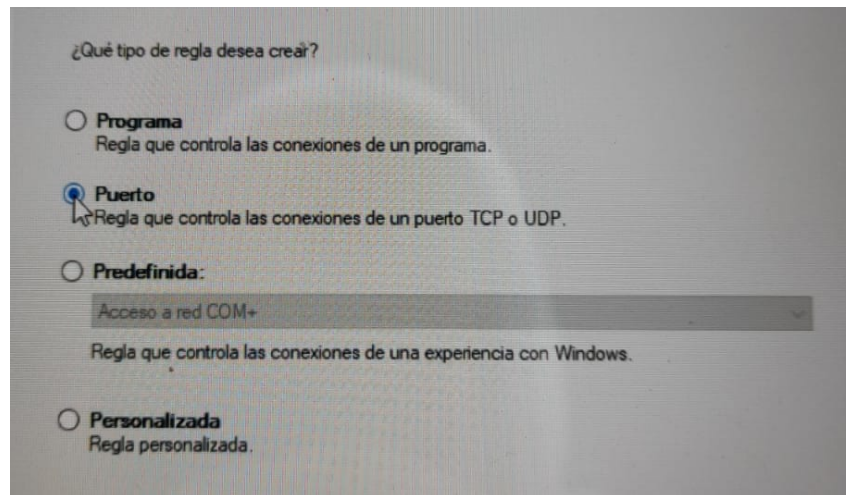


Figure 3: Seleccionamos que la regla controla conexiones a un puerto

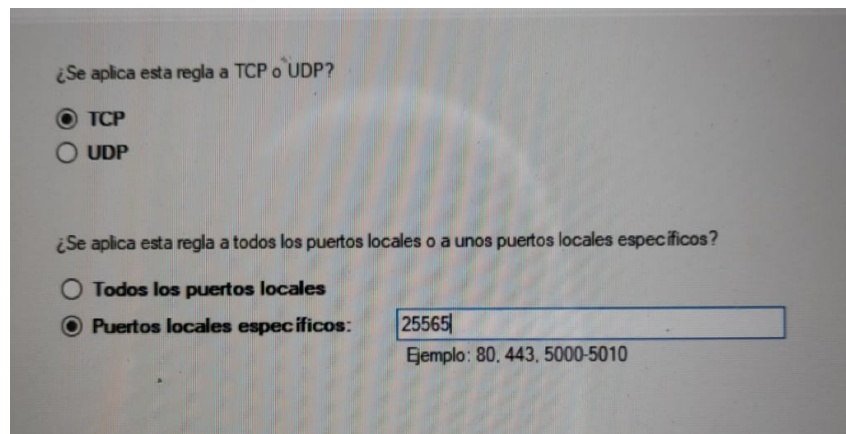


Figure 4: Establecemos el puerto 25565 como el puerto que va a controlar la regla.

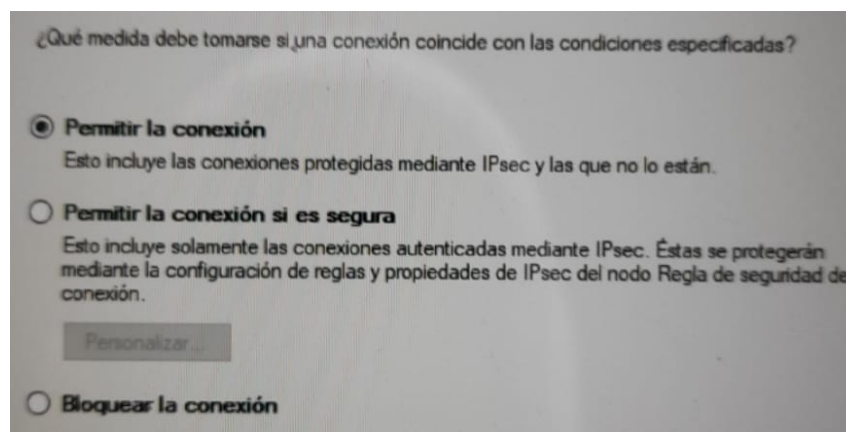


Figure 5: Permitimos lass conexiones provenientes, excluyendo conexiones seguras.

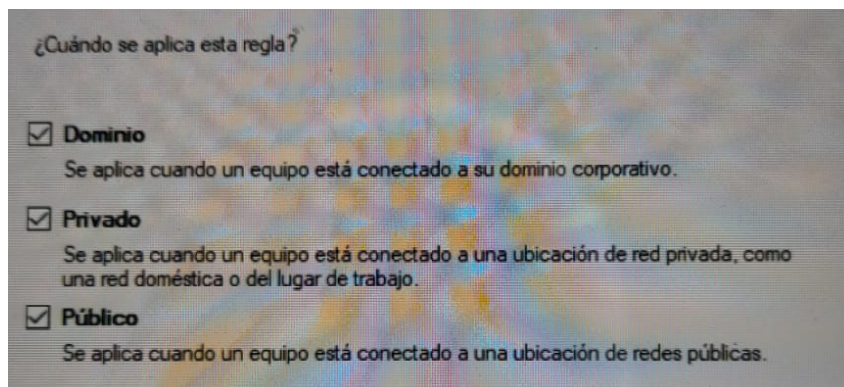


Figure 6: Seleccionamos que vamos a permitir conexiones de ámbito privado, publico y del dominio.

Finalmente establecemos un nombre descriptivo a la regla, y recargamos la configuración del firewall para que se apliquen los cambios. Tras esto, el servidor se encontrará disponible en el puerto 25565 en la dirección IP asociada a la máquina de Windows Server.

5 Benchmark

El benchmarking es una técnica que consiste en comparar el rendimiento de un sistema, producto o servicio con respecto a otros similares en el mercado. Su objetivo principal es identificar las mejores prácticas y oportunidades de mejora, permitiendo a las organizaciones optimizar sus procesos y alcanzar altos niveles de eficiencia.

Esta herramienta se utiliza en diversos ámbitos, como la industria, la tecnología y los negocios, para evaluar el desempeño de diferentes aspectos, como la velocidad, la capacidad, la seguridad y la calidad. Al realizar comparaciones con referentes del mercado, se pueden identificar brechas y establecer metas realistas para la mejora continua.

Para llevar a cabo esta práctica, hemos usado la aplicación “Server Wrecker”. También planteamos el uso de otras aplicaciones tanto para el monitoreo del servidor (“Aikar” que, además, está especializada en servidores minecraft) como la puesta en sobreestimulación del mismo (Jmeter). Estas no se pudieron llevar a la práctica debido a incompatibilidades.

Server Wrecker es un programa diseñado específicamente para someter a un servidor a condiciones extremas de carga y estrés. Su objetivo es evaluar la capacidad de respuesta y resistencia del servidor ante situaciones de alta demanda, lo que permite identificar posibles debilidades o cuellos de botella en el sistema.

Este programa simula una carga intensiva de solicitudes al servidor, generando una gran cantidad de peticiones simultáneas. Esto puede ayudar a los administradores de sistemas a determinar los límites de rendimiento del servidor y optimizar su configuración para garantizar un funcionamiento óptimo incluso en momentos de alta demanda.

A continuación vamos a describir los distintos parametros que hemos usado para calcular el rendimiento del sistema y los distintos parámetros.

	Peticiones	Aciertos	Fallos	Tiempo (s)	% fallo	Promedio de CPU	Promedio de RAM (KiB)	Info adicional
Ubuntu Server	10	10	0	18	0%	5,40%	1.127.142	
	50	50	0	40	0%	6,57%	1.138.172	
	100	100	0	190	0%	7,44%	1.263.622	
	200	135	65	513	33%	9,08%	1.018.934	275s server fail
	250	141	109	700	43%	13,73%	1.399.371	270s server fail
	10	10	0	20	0%	15,70%	2.483.086	
Windows Server	50	50	0	141	0%	16,32%	2.492.156	
	100	99	1	212	1%	24,28%	2.585.593	*
	200	105	95	391	47,50%	35,59%	2.698.614	246s server fail
	250	107	143	532	75,20%	40,57%	2.759.165	242s server fail
	* el fallo se produjo en la última conexión							

Figure 7: Datos obtenidos de la realización del benchmark

Las filas en la tabla anterior son las siguientes:

1. Selección de sistema operativo
2. N° de usuarios unidos en un tiempo dado
3. N° de usuarios unidos y mantenidos en el servidor
4. N° de usuarios unidos y que han sido expulsados por el servidor
5. Tiempo empleado para la unión de un número de usuarios
6. Porcentaje de usuarios expulsados de los totales.
7. Tiempo de expulsion masiva de usuarios

5.1 Resultados

Como se ha comentado anteriormente, los parametros que hemos utilizado para hacer las mediciones han sido el uso de CPU, el uso de memoria del sistema, el numero de peticiones hechas al servidor, tanto como aceptadas como fallidas, y el tiempo que ha tardado en completarse el benchmark.

Estas pruebas se han repetido tanto en Ubuntu Server como en Windows Server de manera igualitaria y los resultados han sido los siguientes

5.1.1 Uso de CPU

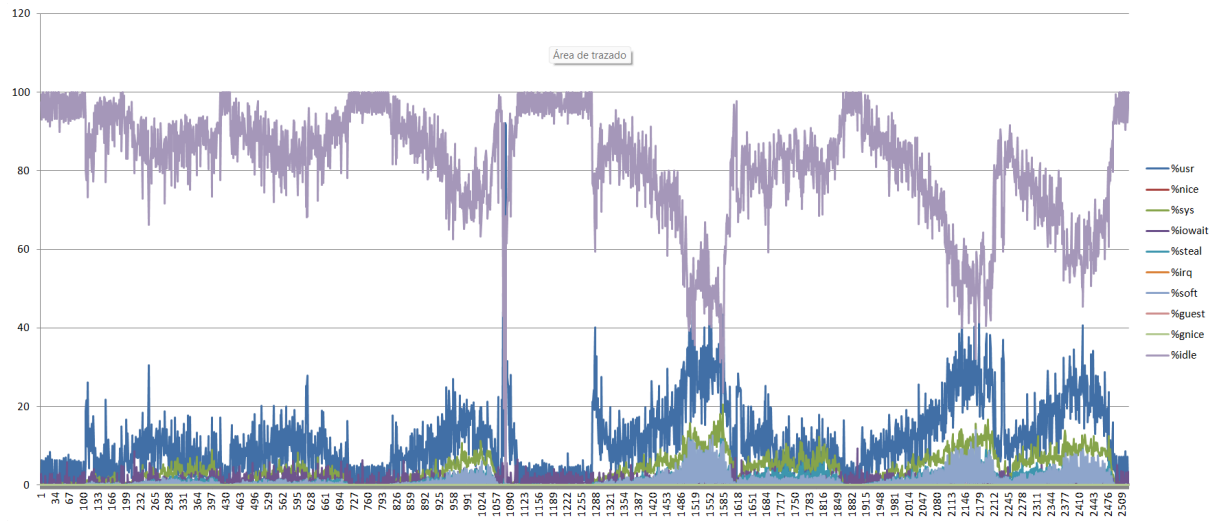


Figure 8: Uso de CPU bajo Ubuntu Server

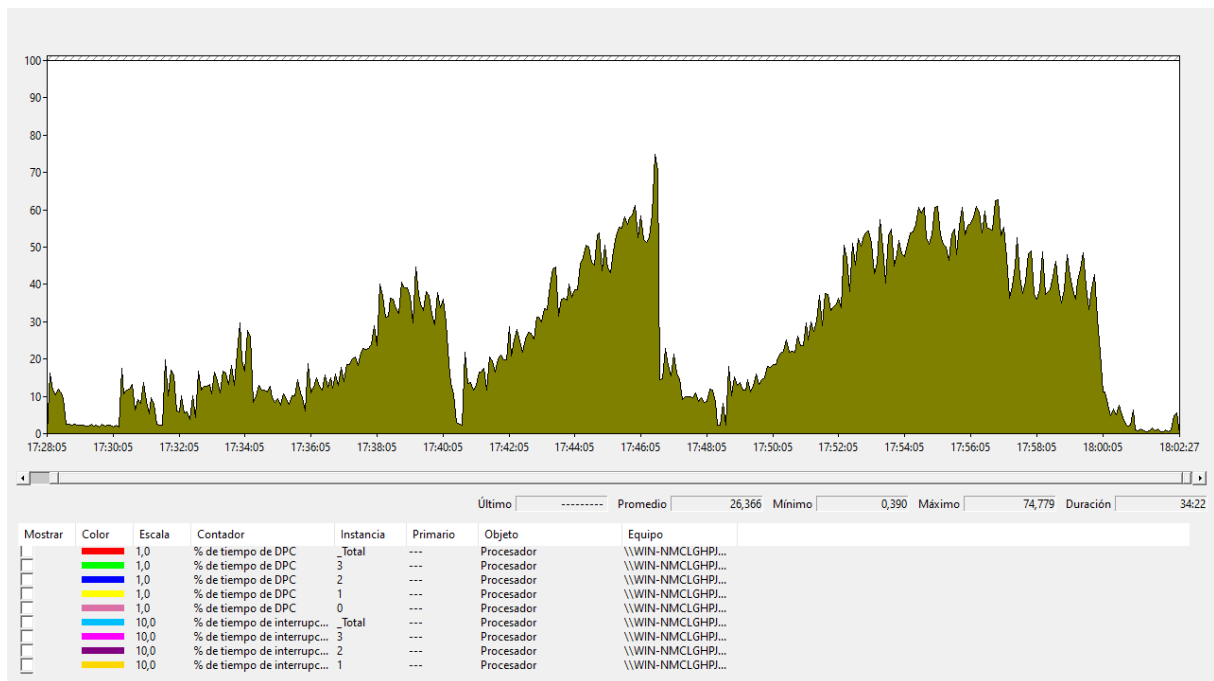


Figure 9: Uso de CPU bajo Windows Server

El uso de CPU está relacionado hasta cierto grado con la cantidad de usuarios en el servidor en un momento dado. Esto es un comportamiento normal, ya que en el caso de un servidor de Minecraft, es responsabilidad del servidor el mantener el estado actual del jugador, con su inventario, posición y movimiento en el mapa.

En el gráfico se pueden apreciar también los distintos intervalos del benchmark realizado, pero nunca llegando a ser más del 50%.

5.1.2 Uso de memoria RAM

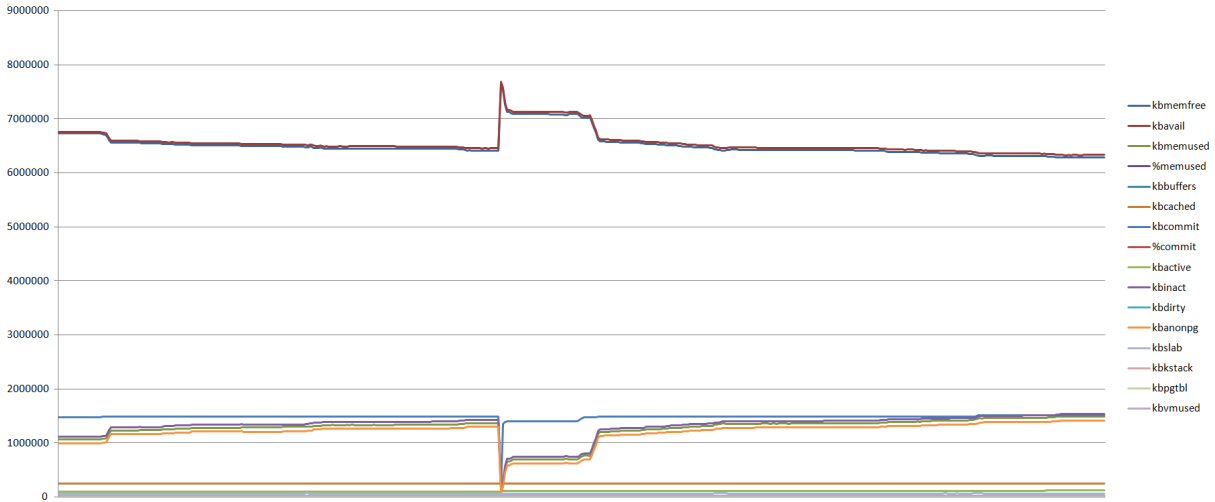


Figure 10: Uso de memoria RAM bajo Ubuntu Server

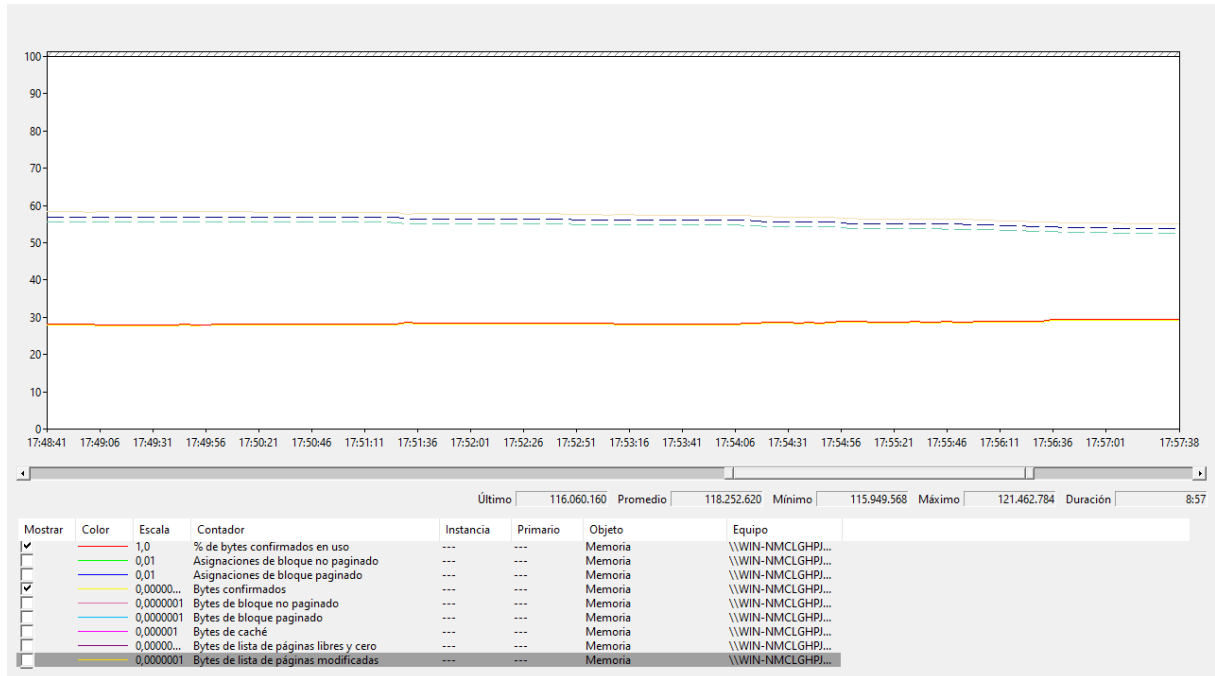


Figure 11: Uso de CPU bajo Windows Server

Observaciones: Durante la prueba el uso de memoria RAM no ha variado ya que la máquina virtual de Java que usa el servidor de Minecraft reserva memoria durante el arranque y luego este no varía en ningún momento.

Cabe destacar una pequeña discrepancia en el uso de memoria durante el benchmark del servidor de Linux debido a que durante la ejecución del benchmark, se cerró la terminal del sistema durante la ejecución.

5.1.3 Número de peticiones

A continuación, vamos a detallar más información sobre las peticiones hechas al servidor. Tenemos tres tipos de peticiones:

- Peticiones fallidas
- Peticiones aceptadas
- Peticiones erroneas

En el momento de consideración de las peticiones fallidas, hemos considerado una petición fallida a una petición que ha provocado la salida de un usuario previamente conectado al servidor.

Durante el benchmarking, debido a las limitaciones del software y la manera que funciona el servidor de Minecraft, en un principio todas las peticiones lanzadas sobre este han sido aceptadas, pero a costa de conexiones ya existentes en el servidor, lo que produce que una conexión nueva expulse a un usuario nuevo.

En total hemos lanzado 610 peticiones divididas en distintos intervalos. Durante la evaluación, nos hemos dado cuenta que Windows Server tiene una mayor tasa de peticiones fallidas comparada con Ubuntu Server. Esto está también relacionado con el uso de recursos de las máquinas durante el benchmark.

En Windows Server, de las 610 peticiones lanzadas en total, 239 han sido catalogadas como peticiones fallidas. Estas peticiones fallidas también están relacionadas con el momento en el que el servidor supera las 107 conexiones activas en un momento dado.

En Ubuntu Server, de las 610 peticiones lanzadas en total, 174 han sido catalogadas como peticiones fallidas. En el caso de esta máquina, se han empezado a considerar las peticiones como fallidas a partir de las 141 conexiones aproximadamente.

5.1.4 Duración del benchmark

Durante la ejecución del benchmark, tomando en consideración los datos del numero de peticiones lanzadas en un momento dado, podemos observar que el numero de peticiones fallidas y la duración del benchmark están directamente relacionadas de forma lineal.

Es decir, a mayor cantidad de conexiones son fallidas, la duración del benchmark dura menos. Esto no sabemos exactamente por que ha sido así. Puede ser debido a que a medida que las conexiones van aumentando, el servidor tarda más en aceptar nuevas conexiones, lo cual aumenta la latencia dentro del servidor y causa que el tiempo de espera exceda el máximo programado, lo cual hace que el cliente se salga del servidor.

Esto juntado con que Linux mantiene más conexiones que Windows, podría significar que el servidor simplemente no puede aceptar más, y antes de denegar conexiones, prefiere intentar mantener las que ya tiene, lo cual produce que el tiempo de espera exceda el máximo programado durante el benchmark.

Otra suposición es que el servidor de Minecraft en Linux, cuando llega a su punto de saturación, este tiene un comportamiento más radical que en Windows, ya que Linux, al llegar a ese punto, antes de aceptar nuevas conexiones, se ha dado el caso de que el servidor ha hechado a la totalidad de todos los usuarios del servidor antes de aceptar una nueva conexión.

Este comportamiento no se ha podido replicar en Windows Server, creemos que es debido a las limitaciones del hardware de la máquina más que un comportamiento del propio servidor de Minecraft.

En total, Windows Server ha tardado un total de 1296 segundos en completar el benchmark, mientras que Ubuntu Server ha tardado 1461 segundos, aunque pensamos que esta medición puede ser considerada anomala y no una medida considerable para hacerse un estudio debido al extraño comportamiento de los servidores.

5.2 Análisis de los datos

A continuación vamos a analizar si ambas máquinas son estadísticamente diferentes. Para ello hemos recurrido a hacer un análisis estadístico en base a la función de distribución t-student con cuatro grados de libertad.

Considerando los datos siguientes

$$\hat{d} = 33; s = 23.06$$

Partiendo de un valor de t-student calculado a través de las fórmulas proporcionadas

$$t = 3.2$$

Y tomando valor de cuatro grados de libertad, como ha sido mencionado anteriormente, la probabilidad de obtener un valor de t superior a 3.2 es:

$$P = 3.29\%$$

¿Esto es mucho, o es poco? Definiendo el nivel de significatividad como

$$\alpha = 0.05$$

obtenemos el intervalo de confianza, que es

$$[-2, 78; 2, 78]$$

como t_{Exp} no se encuentra en el rango, concluimos que las máquinas no pueden ser estadísticamente iguales con un 95% de confianza.

Es decir, podemos demostrar que existe una diferencia en el rendimiento comparada entre Windows Server y Ubuntu Server.

6 Observaciones y conclusión

En este estudio se ha evaluado el rendimiento de un servidor de Minecraft en dos sistemas operativos diferentes, Windows Server y Ubuntu Server. Se ha realizado la instalación y configuración de ambos sistemas operativos y se ha llevado a cabo un benchmark para comparar el uso de CPU, memoria RAM, número de peticiones y duración del benchmark entre ambos sistemas operativos.

Dados los resultados que se han obtenido, se han revelado algunas diferencias significativas en el rendimiento entre Windows Server y Ubuntu Server. En general, ha podido apreciar que Ubuntu Server presentaba un uso más eficiente de recursos en términos de uso de CPU y memoria RAM. Además, se encontró que Ubuntu Server era capaz de manejar un mayor número de peticiones simultáneas en comparación con Windows Server.

Sin embargo, se debe tener en cuenta que estos resultados pueden variar dependiendo de las especificaciones técnicas de la máquina utilizada y la configuración específica del servidor de Minecraft. Por lo tanto, es importante realizar pruebas adicionales en diferentes entornos para obtener conclusiones más sólidas.

En cuanto a un plan a futuro, se sugiere continuar explorando otras alternativas de sistemas operativos, como por ejemplo otras distribuciones de Linux y realizar pruebas adicionales para obtener una visión más completa del rendimiento del servidor de Minecraft.

En resumen, este estudio proporciona una evaluación inicial del rendimiento del servidor de Minecraft en Windows Server y Ubuntu Server. Los resultados sugieren que Ubuntu Server puede ofrecer un mejor rendimiento en términos de uso de CPU, memoria RAM y capacidad para manejar un mayor número de peticiones simultáneas.

References

- [1] AlexProgrammerDE - ServerWrecker [<https://github.com/AlexProgrammerDE/ServerWrecker>]
- [2] Overleaf - Online L^AT_EX editor tutorials [<https://www.overleaf.com/learn/latex/Tutorials>]
- [3] Cyberclick - Marketing digital (2022) Plan de Marketing - Qué Es y Cómo Hacerlo (Curso 2023) <https://www.youtube.com/watch?v=1gu8RrD48BA&t=200s> (con marca temporal)

- [4] Monica <https://monica.im/>
- [5] Ricrafdo (2019) COMO HACER UN SERVIDOR DE MINECRAFT EN TU PC Y CONFIGURARLO <https://www.youtube.com/watch?v=brJjEiBitL0>
- [6] Get Ubuntu Server <https://ubuntu.com/download/server/choose>
- [7] Microsoft (2022) Windows Server 2022 Centro de Evaluación <https://www.microsoft.com/es-es/evalcenter/download-windows-server-2022>
- [8] Debian Wiki (2023) KVM <https://wiki.debian.org/KVM>
- [9] Tecadmin (2023) Setting Up a Port Forwarding Using Iptables in Linux <https://tecadmin.net/setting-up-a-port-forwarding-using-iptables-in-linux/>